Jessica Wu
117966510

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    var count: int := a.Length-1;
    out := 0;

    while (count >= 0) {
        out := out - a[count];
        count := count - 1;
    }

}
```

```
method f2(a:array<int>) returns (out:int) {
    var count: int := 0;
    out := 0;

    while (count < a.Length) {
        out := out - a[count];
        count := count + 1;
    }

}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`

a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`

a → a → b

(d) `\x l -> x : l ++ l ++ [x]`

a → [b] → [a, b]

(e) `foldr (const l)`

a → [b] → a

(f) `(."42")`

(a, b) → (a, b)

(g) `(.) "42"`

((a, b)) → ((a, b))

(h) `reverse . reverse`

[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`

[a] → [a]

(j) `let f x = x in (f 'a', f True)`

(a → b) → c → c

(k) `fmap (fmap (+1)) [Nothing]`

[Maybe a] → [Maybe a]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

```
\x -> if x then [True] else [False]
```

(c) `a -> Maybe b`

```
\a -> Nothing
```

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

```
f x y -> if f x y then True else False
```

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

```
f x y -> Just c
```

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

```
\[x] [y] -> f[x]
```

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

```
\a -> Nothing
```

(h) `Eq a => a -> [a] -> [a]`

```
\x -> x : [x]
```

(i) `Show a => [a] -> IO String`

```
\x -> x + "hi"
```

(j) `(a,b) -> (a -> b -> c) -> c`

```
\(x,y) -> f x y
```

(k) `((a,b) -> c) -> c`

```
\x -> f x
```

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m >= 0 && 0 = m*m*(m-m) + m*m                    }}
  y := 0;
{{ m >= 0 && y = m*m*(m-m) + m*m                    }}
  var x := m * m;
{{ m >= 0 && y = x*(m-m) + x                        }}
  var z := m;
{{ z >= 0 && y = x*(m-z) + x                        }}
  while (z > 0) {
{{ z >= 0 && y = x*(m-z) + x && z > 0               }} ->>
{{ z-1 >= 0 && y+x = x*(m-(z-1)) + x                }}
    z := z - 1;
{{ z >= 0 && y+x = x*(m-z) + x                      }}
    y := y + x;
{{ z >= 0 && y = x*(m-z) + x                        }}
}
{{ z >= 0 && y = x*(m-z) + x && !(z > 0)            }} ->>
{{ y = m * m * m}}
```

m = 4
y = 0
x = 16
z = 4

z	x	y
4	16	16
3	16	32
2	16	48
1	16	64

y = x · (m - z + 1)

y = x · (m - z) + x

y =

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave _ list = list
weave list _ = list
weave (x:xs) (y:ys) = [x,y]: weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset [] = []
powerset (x:xs) = (x:xs): powerset x : powerset xs
```

Wasis Ali
115713178

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
var count := a.length-1;
var acc := 0;
while (count >= 0) {
acc := acc - a[count];
count := count - 1;
}
out := acc;
}
```

```
method f2(a:array<int>) returns (out:int) {
var count := 0;
var acc := 0;
while (count <= a.length-1) {
acc := acc - a[count];
count := count + 1;
}
out := acc;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
x -> y -> (x, y)

(c) `\x y -> if x == y then show x else show (x,y)`

(d) `\x l -> x : l ++ l ++ [x]`

(e) `foldr (const 1)`
(a -> b -> a) -> [a]

(f) `(,"42")`

(g) `(,) "42"`
ill-type

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
[a] -> [a]

(j) `let f x = x in (f 'a', f True)`

(k) `fmap (fmap (+1)) [Nothing]`
~~([Int] -> (Int -> Int -> Int) -> [Int])~~
((Int -> Int -> Int) -> [Int] -> [Int]) -> [
ill-type

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\b -> b : []`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f is cs -> zipWith f is cs`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{                                          }}
  y := 0;
{{                                          }}
  var x := m * m;
{{                                          }}
  var z := m;
{{                                          }}
  while (Z > 0) {
{{                                          }} ->>
{{                                          }}
    z := z - 1;
{{                                          }}
    y := y + x;
{{                                          }}
}
{{                                          }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [b] -> [c]
weave x:xs y:ys = x : y : weave xs ys
weave _ _ = []
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
Powerset :: [a] -> [[a]]
Powerset x:xs = x:xs : powersethelp x xs :
  Powerset xs

Powersethelp :: a -> [a] -> [[a]]
Powersethelp x y:ys = [x : y] : Powersethelp x ys
Powersethelp x _ = [x]
```

Alex Lee

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0;
  var i := a.Length-1;
  while i >= 0
  {
    out := a[i] - out;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  out := 0;
  var i := 0;
  while i < a.Length
  {
    out = out - a[i];
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
ill-typed

(f) `(,"42")`
ill-typed

(g) `(,) "42"`
b → (String, b)

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
(a, b)

(k) `fmap (fmap (+1)) [Nothing]`
ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> true : (repeat x)`

(c) `a -> Maybe b`
undefined

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f i c -> true : (zipWith f (5: i) ('a' : c))`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f (Just x) (Just y) -> Just (f x y)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\.`

(h) `Eq a => a -> [a] -> [a]`
`Eq x [] = [x]`
`Eq x y = y`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`
`\f -> f (, )`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 == (m-m)·m·m ∧ m>=0 ∧ m·m == m·m                    }}
  y := 0;
{{ y == (m-m)·m·m ∧ m>=0 ∧ m·m == m·m                    }}
  var x := m * m;
{{ y == (m-m)·x ∧ m>=0 ∧ x == m·m                        }}
  var z := m;
{{ y == (m-z)·x ∧ z>=0 ∧ x == m·m                        }}
  while (Z > 0) {
{{ y == (m-z)·x ∧ z>=0 ∧ x == m·m ∧ z > 0                }} ->>
{{ y + x == (m-(z-1))·x ∧ z-1 >= 0 ∧ x == m·m            }}
    z := z - 1;
{{ y + x == (m-z)·x ∧ z>=0 ∧ x == m·m                    }}
    y := y + x;
{{ y == (m-z)·x ∧ z>=0 ∧ x == m·m                        }}
}
{{ y == (m-z)·x ∧ z>=0 ∧ x == m·m ∧ !(z > 0)             }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] _ = []
weave [] _ = []
weave (x:xs) (y:ys) = [x] ++ [y] ++ (weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset [] = [[]]
powerset (x:xs) = (x:(x:xs)) : ((x:xs) : (powerset xs)))
(powerset (tail (x:xs)))
```

Erik Pfeffer

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

[1, 2, 3]
3-0=3
2-3=-1
1-(-1)=2

[1,2,3]
0-3=-3
-3-2=-5
-5-1=-6

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out:=0;
  var i:=a.Length-1;
  while(i>=0){
    out:=a[i]-out;
    i:= i-1;
  }
  return;
}
```

```
method f2(a:array<int>) returns (out:int) {
  out:=0;
  var i:=0;
  while(i<a.Length){
    out:= out-a[i];
    i:= i+1;
  }
  return;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const l)`
b → [a] → Int

(f) `(,"42")`
a → String → (a, String)

(g) `(.) "42"`
a → String → (a, String)

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`
ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\a -> [a && False]`

(c) `a -> Maybe b`
`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`zipWith (\a b -> a > 2 || b /= 'c')`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`zipWith (\(Just a) (Just b) -> Just (a,b))`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`f f1 f2 a l = map (f2 . f1) a l`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`foo = undefined`

(h) `Eq a => a -> [a] -> [a]`
`\a -> a l -> if a == a then a:a l else a l`

(i) `Show a => [a] -> IO String`
`putStrLn`

(j) `(a,b) -> (a -> b -> c) -> c`
`f (a,b) fn = fn a b`

(k) `((a,b) -> c) -> c`
`f = undefined.`

m:3, z>0
x:9, y:0, z:3
9, 9, 2
9, 18, 1
9 27, 0

y=m·m·m
y = m · m · (m-z)

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

m = 2 , z>0
x=4, y=0, z=2
4, 4, 1
9, 8, 0
z>=0

m=4, z>0
x=16, y=0, z=m=4
16, 16, 3
, 32, 2
48, 1
64, 0

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m>=0 && 0 = m·m·(m-m)                                  }}
  y := 0;
{{ m>=0 && y = m·m·(m-m)                                  }}
  var x := m * m;
{{ m>=0 && y = m·m·(m-m)                                  }}
  var z := m;
{{ z>=0 && y = m·m·(m-z)                                  }}
  while (Z > 0) {
{{ z>=0 && y = m·m·(m-z) && z>0                           }} ->>
{{ z-1>=0 && y+z = m·m·(m-(z-1))                          }}
    z := z - 1;
{{ z>=0 && y+x = m·m·(m-z)                                }}
    y := y + x;
{{ z>=0 && y = m·m·(m-z)                                  }}
}
{{ z>=0 && y = m·m·(m-z) && !(z>0)                        }} ->>
{{ y = m * m  * m}}
```

[1,2,3] (1:[2,3]
[1,2,3]:

Question 5 (20 points)

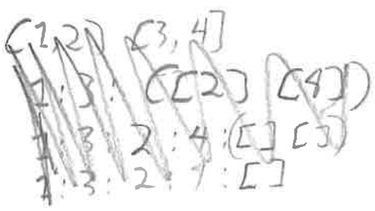
Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] _ = []
weave _ [] = []
weave (x:xs) (y:ys) = x:y:(weave xs ys)
```



(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

[1,2,3]

```
powerset (x:xs) = [] : powersetAux x xs where
  powersetAux :: a -> [a] -> [[a]]
  powersetAux e [] = [[e]]
  powersetAux e [y] = [e]:[y]:[[e,y]]
  powersetAux e (y:ys) = [e]:(y:ys):(powersetAux e ys):(powersetAux y ys)
```

1 [1]

```
powersetAux e (y:ys) = e:y:ys:(powersetAux e ys):
                       1  2  []
                       [1,2],[1]
                       (powersetAux y ys)
                       ++[e,y]
```

Aidan Caruso

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    var i:int := a.Length-1
    out:= 0
    while(i>=0) {
        out := out - a[i]
        i := i-1
    }
}
```

```
method f2(a:array<int>) returns (out:int) {
    var i:int := 0
    out := 0
    while (i < a.Length) {
        out:= out - a[i]
        i := i+1
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
(Eq a, Show a, Show b) => a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const 1)`
ill-typed

(f) `(."42")`
(), String

(g) `(.) "42"`
ill typed

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] -> [Int]

(j) `let f x = x in (f 'a', f True)`
ill typed

(k) `fmap (fmap (+1)) [Nothing]`
ill typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [true] else [false]`

(c) `a -> Maybe b`
Nothing

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f a b -> true : (zipWith (\a b -> f a b) a b)`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f a b ->`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f1 f2 as -> filter f2 $ map f1 as`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
~~map f a f ->~~ undefined

(h) `Eq a => a -> [a] -> [a]`
`\a as -> if a == a then as else as`

(i) `Show a => [a] -> IO String`
`f = putStr(lm) $ foldl (\a b -> show a ++ b) ("")`

(j) `(a,b) -> (a -> b -> c) -> c`
`f (a,b) f2 = f2 a b`

(k) `((a,b) -> c) -> c`
`\f ->` undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = (m-m)·m*m ∧ m>0 ∧ m*m = m*m }}
  y := 0;
{{ y = (m-m)·(m*m) ∧ m>0 ∧ m*m = m*m }}
  var x := m * m;
{{ y = (m-m)·x ∧ z>0 ∧ x = m*m }}
  var z := m;
{{ y = (m-z)·x ∧ z>0 ∧ x = m*m }}
  while (z > 0) {
{{ y = (m-z)·x ∧ z>0 ∧ x = m*m }} ->>
{{ y+x = (m-(z-1))·x ∧ z>0 ∧ x = m*m }} ->>
    z := z - 1;
{{ y+x = (m-z)·x ∧ z>0 ∧ x = m·m }}
    y := y + x;
{{ y = (m-z)·x ∧ z>0 ∧ x = m*m }}
  }
{{ y = (m-z)·x ∧ z>0 ∧ z<=0 ∧ x = m*m }} ->>
{{ y = m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] _ = []
weave _ [] = []
weave (x:xs) (y:ys) = [x] ++ [y] ++ weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = []
powerset (x:xs) = let p = powerset xs in
                  map (x:) p ++ p
```

Abbas Islam

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length;
  out := 0;
  while (i >= 0) {
    out := out - a[i];
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → b → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
b → [a] → a

(f) `(,"42")`
ill-typed

(g) `(,) "42"`
String → [String]

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
(a → Bool)

(k) `fmap (fmap (+1)) [Nothing]`
ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [True] else [False]`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\a (2:xs) ('c':ys) -> if (a 2 'c') then [True] else [False]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\a b c -> liftA2 a b c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`undefined`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\(Just a) c -> if c a == [] then [(a,`

(h) `Eq a => a -> [a] -> [a]`
`\a -> (==) a []`

(i) `Show a => [a] -> IO String`
`\a -> print a`

(j) `(a,b) -> (a -> b -> c) -> c`
undefined

(k) `((a,b) -> c) -> c`
undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules– write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 == (m-m)(m*m) && m >= 0 && m*m == m*m }}
  y := 0;
{{ y == (m-m)(m*m) && m >= 0 && m*m == m*m }}
  var x := m * m;
{{ y == (m-m)(x) && m >= 0 && x == m*m }}
  var z := m;
{{ y == (m-z)(x) && z >= 0 && x == m*m }}
  while (z > 0) {
{{ y == (m-z)(x) && z >= 0 && (z > 0) && x == m*m }} ->>
{{ y+x == (m-(z-1))(x) && z-1 >= 0 && x == m*m }}
    z := z - 1;
{{ y+x == (m-z)(x) && z >= 0 && x == m*m }}
    y := y + x;
{{ y == (m-z)(x) && z >= 0 && x == m*m }}
  }
{{ y == (m-z)(x) && z >= 0 && z <= 0 }} ->>
{{ y = m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1;2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave a b = case (a,b) of
  ([],_) -> []
  (_,[]) -> []
  (x:xs, y:ys) -> (x:[y]) ++ (weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset lst = powerset_aux lst []
```

```
powerset_aux lst acc = case lst of
  [] -> []:acc
  (x:xs) -> if [x] in acc then (powerset_aux xs acc) else if
            xs in acc then (powerset_aux xs xs:acc) else
            powerset_aux xs ([x]:acc)
```

Luke Zic
11863643 6

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i:=a.Length-1;
  out:=0;
  while i>=0 {
    out:=a[i]-out;
    i:=i-1;
  }
  return out;
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i:=0;
  out:=0;
  while i<a.Length {
    out:=a[i]-out;
    i:=i+1;
  }
  return out;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
String → String → String

(d) `\x y -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const l)`
ill-typed

(f) `(."42")`
ill-typed

(g) `(."42"`
ill-typed

(h) `reverse . reverse`
ill-typed

(i) `\l -> filter (< 42) (l ++ l)`
[a] → [a]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`
ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> case x of [_] -> [x]`

(c) `a -> Maybe b`
`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\x y z -> if fold1 - 0 y then if fold1 + z then [True] else [False] else [False]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\x y z -> If a /= Nothing && b /= Nothing then just x y z else just x y z`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\x y z -> case z of [a] -> If y (x a) then [x a]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\x y -> [(x, case (y (x)) of [b] -> b)]`

(h) `Eq a => a -> [a] => [a]`
`\x y -> case`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`
`\x y -> case x [(a,b)] -> y a b`

(k) `((a,b) -> c) -> c`
`\x -> y`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{   m*m = (m-1)*m*m && z-1 >= 0                    }}
  y := 0;
{{ y+m*m = (m-1)*m*m && z-1 >= 0                    }}
  var x := m * m;
{{ y+x = (m-1+m)*m*m && z-1 >= 0                    }}
  var z := m;
{{ y+x = (z-1+m)*m*m && z-1 >= 0                    }}
  while (z > 0)
{{ y+x = (z-1)*m*m && z-1 >= 0                      }} ->>
{{ y+x = (z-1+m)*m*m && z-1 >= 0                    }}
    z := z - 1;
{{ y+x = (z+m)*m*m && z >= 0                        }}
    y := y + x;
{{ y = (z+m)*m*m && z >= 0                          }}
}
{{ y = (z+m)*m*m && !(z > 0)                        }} ->>
{{ y = m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
let weave lst1 lst2 =
  case lst1, lst2 of
  ([],[]) -> []
  (a:b, e:f) -> a:e : weave b f
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
let Powerset lst = ???
  case lst of
  [] -> [[]]
  a:b -> [a]:Powerset b: Powerset b++[a]
```

Siddhant Kumar

1 1 9 3 1 1 7 4 5

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)
```

-> accumulated

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0
  var i := int := a.Length - 1
  while (i >= 0) {
  out := a[i] - out
  i := i-1
  }
  return out
}
```

```
method f2(a:array<int>) returns (out:int) {
  out := 0
  var i := int := 0
  while (i < a.Length) {
  out := out - a[i]
  i := i+1
  }
  return out
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
Eq => a Show => a a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
ill typed

(f) `(,"42")`
a → (a, Int)

(g) `(.) "42"`
a → (Int, a)

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[int] → [int]

(j) `let f x = x in (f 'a', f True)`
ill typed

(k) `fmap (fmap (+1)) [Nothing]`
[Maybe [Int]] → Maybe ([Int])

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then x : [True] else x : [False]`

(c) `a -> Maybe b`
`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f2 l1 l2 -> map (\(a,b) -> f2 a b) $ zip l1 (l2 ++ ["c"])`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f a b -> if isNothing a || isNothing b then Nothing else f (head $ Maybe to list a) (head $ Maybe to list b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f1 f2 l -> filter f2 (map f1 l)`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\v f1 -> if isMaybe v then (val, fval) : else [(a,)]`
where `val = head (Maybe to list v)`

(h) `Eq a => a -> [a] -> [a]`
`\v l -> filter ((==) v) l`

(i) `Show a => [a] -> IO String`
`\l -> putStr (foldr (\e a -> show e ++ a) "" l)`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`
`\ ill typed =`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

$m \times m(m-1) + m$

$\rightarrow \quad x * z + y = m * m * m$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m*m*m + 0 = m*m*m  /\  m*m = m*m  }} /\ (m > 0)
  y := 0;
{{ (m*m)*m + y = m*m*m  /\  m*m = m*m }} /\ (m > 0) }
  var x := m * m;
{{ x*m + y = m*m*m  /\  x = m*m  /\ m>=0}
  var z := m;
{{ x*z + y = m*m*m  /\  x = m*m  /\ (z >= 0)
  while (Z > 0) {
{{ x*z+y = m*m*m  /\ x = m*m /\ (z>=0)}} ->> (z > 0)
{{ x*(z-1) + y + x = m*m*m  /\ x = m*m /\ (z-1 >= 0)
    z := z - 1;
{{ x*(z) + (y+x) = m*m*m  /\ x = m*m /\ (z >= 0)
    y := y + x;
{{ x*z + y = m*m*m  /\ x = m*m /\ (z >= 0)}}
}
{{ x*z + y = m*m*m  /\ x = m*m }} ->>
{{ y = m * m  * m}}        /\ (z >= 0) /\ !(z > 0)}}
```

$Inv = \{(x * z + y = m * m * m) \wedge x = m * m \wedge z \geq 0\}$

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] [] = []
weave (x:xs) (y:ys) = (x : y) : (weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]
powerset (x:xs) = group (x:xs) ++ powerset (xs)
```

Dashawna Lara
117916760

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i:int := a.Length-1;
  out := 0;
  while(i >= 0 ) {
    out := out - a[i];
    i := i - 1;
  }
}

}
```

```
method f2(a:array<int>) returns (out:int) {
  var i:int := 0;
  out := 0;
  while(i < a.Length;
    out := out - a[i];
    i := i + 1;
  }

}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`

a → b → (a , b)

(c) `\x y -> if x == y then show x else show (x,y)`

ill-typed

(d) `\x l -> x : l ++ l ++ [x]`

a → [a] → [a]

(e) `foldr (const 1)`

const 1 = a → b → b

a → [a] → a

(f) `(,"42")`

a → (a)

(g) `(,) "42"`

a → (a)

(h) `reverse . reverse`

(.) :: (b → c) → (a → b) → a → c
([b] → [b]) → ([a] → [a]) → [a] → [b]

[a] → [b]

(i) `\l -> filter (< 42) (l ++ l)`

Ord a ⇒ [a] → [a]

(j) `let f x = x in (f 'a', f True)`

(char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`

fmap :: (a → b) → f a → f b

a → Maybe a

fmap (+1) :: a → f(a)

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`\x -> [x && true]`

(c) `a -> Maybe b`

`\x -> Just "idk"`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`zipwith (\x,c -> True) [1,2,3] ['a','b','c']`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`zipwith`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\x, b, (a:as) -> [x a]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\Maybe x, b -> (s:xs) = bx in [(x,s)]`

(h) `Eq a => a -> [a] -> [a]`

`delete`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

possible invariants:

$z \geq 0$,

$y = (m-z) \cdot x$

$6-6 \cdot 12 = 0$

$6-5 \cdot 12 = 12$

$6-4 \cdot 12 = 24$

$6-3 \cdot 12 = 36$

$m = 6$:

y	x	z
0	12	6
12	12	5
24		4
36		3
48		2
60		1
72		0

$y =$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules– write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = (m-m)·(m·m) && m > 0 && m != 0                    }}
  y := 0;
{{ y = (m-m)·(m·m)  && m > 0 && m != 0                   }}
  var x := m * m;
{{ y = (m-m)·x      && m > 0 && m != 0                   }}
  var z := m;
{{ y = (m-z)·x      && z > 0 && m != 0                   }}
  while (z > 0) {
{{ y = (m-z)·x      && z > 0 && z > 0 && m != 0          }} ->>
{{ (y+x) = (m-(z-1))·x && (z-1) > 0 && m != 0            }}
    z := z - 1;
{{ (y+x) = (m-z)·x  && z > 0 && m != 0                   }}
    y := y + x;
{{ y = (m-z)·x      && z > 0 && m != 0                   }}
}
{{ y = (m-z)·x      && z > 0 && m != 0 && !(z > 0)       }} ->>
{{ y = m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] _ = []
weave _ [] = []
weave (x:xs)(y:ys) = [x]++[y]++(weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

powerset

Orion Daniel

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i : int := a.Length;
  out := 0;
  while (i < 0) {
    out := out - a[i];
    i := i - 1;
  }
  return out;
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i : int := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
  }
  return out;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`

(c) `\x y -> if x == y then show x else show (x,y)`

(d) `\x l -> x : l ++ l ++ [x]`

(e) `foldr (const l)`
b -> [a] -> b

(f) `(."42")`
ill-typed

(g) `(.) "42"`
Char -> b -> (Char, b)

(h) `reverse . reverse`
([a] -> [a]) -> ([b] -> [b]) -> [b] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
(a -> Bool) -> [a] -> [a] -> [a]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`
(a -> b) -> (a -> b) -> Int -> Int -> [Maybe]

(a -> Bool) -> [a] -> [a]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
repeat True

(c) `a -> Maybe b`
a = Nothing

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
ZipWith (1, 'a', True)

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
'ZipWith listToMaybe

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
zip (Maybe a)

(h) `Eq a => a -> [a] -> [a]`
group (:) a

(i) `Show a => [a] -> IO String`
show [Char]

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 + m*m = m*m*m && m > 0                    }}
  y := 0;
{{ (y + (m*m)) = m*m*m && m > 0                }}
  var x := m * m;
{{ (y + x) = m*m*m && m > 0                    }}
  var z := m;
{{ (y + x) = m*m*m && z > 0                    }}
  while (z > 0) {
{{ y = m*m*m && z > 0                          }} ->>
{{ (y + x) = m*m*m && (z-1) > 0                }}
    z := z - 1;
{{ (y + x) = m*m*m && z > 0                    }}
    y := y + x;
{{ y = m*m*m && z > 0                          }}
  }
{{ y = m*m*m && !(z > 0)                       }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
function weave a b {
z = case (a,b) of
        ([],E]) -> []
        (x:xs, y:ys) -> x:y: weave xs ys
return z
```

}

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
function powerset a {
    foldr (\x -> (y:ys) -> y ++ (powerset ys)) a;
```

}

1 Hoare Rules

```
  { P } s1 { Q }
  { Q } s2 { R }
------------------- (hoare_seq)
{ P } s1; s2 { R }


------------------------ (hoare_asgn)
{Q [x := a]} x := a {Q}


  {P'} c {Q'}
P ==> P'    Q' ==> Q
-------------------- (hoare_consequence)
   {P} c {Q}

         {P &&  b} s1 {Q}
         {P && !b} s2 {Q}
-------------------------------- (hoare_if)
{P} if b { s1 } else { s2 } {Q}

         {P && b} s {P}
---------------------------- (hoare_while)
{P} while b { s } {P && !b}
```

Functors, Applicatives, and Monads

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$) :: Functor f => a -> f b -> f a
($>) :: Functor f => f a -> b -> f b

liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
(*>) :: Applicative f => f a -> f b -> f b
(<*) :: Applicative f => f a -> f b -> f a
when :: Applicative f => Bool -> f () -> f ()

(>>) :: Monad m => m a -> m b -> m b
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
filterM :: Monad m => (a -> m Bool) -> [a] -> m [a]
sequence  :: Monad m => [m a] -> m [a]
join :: Monad m => m (m a) -> m a
zipWithM :: Monad m => (a -> b -> m c) -> [a] -> [b] -> m [c]
foldM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m b
replicateM :: Applicative m => Int -> m a -> m [a]

liftM :: Monad m => (a1 -> r) -> m a1 -> m r
liftM2 :: Monad m => (a1 -> a2 -> r) -> m a1 -> m a2 -> m r
...
```

Prelude Functions and Datatypes

Booleans

```
data Bool = False | True

(&&) :: Bool -> Bool -> Bool
(||) :: Bool -> Bool -> Bool
not  :: Bool -> Bool
```

Lists

```
data [] a = [] | a : [a]

(++) :: [a] -> [a] -> [a]
head, last :: [a] -> a
tail, init :: [a] -> [a]

null   :: Foldable t => t a -> Bool
length :: Foldable t => t a -> Int
map :: (a -> b) -> [a] -> [b]
reverse :: [a] -> [a]
intersperse :: a -> [a] -> [a]
transpose :: [[a]] -> [[a]]
foldl  :: Foldable t => (b -> a -> b) -> b -> t a -> b
foldr  :: Foldable t => (a -> b -> b) -> b -> t a -> b
concat :: Foldable t => t [a] -> [a]
and, or  :: Foldable t => t Bool -> Bool
any, all :: Foldable t => (a -> Bool) -> t a -> Bool
sum, product :: (Foldable t, Num a) => t a -> a
maximum, minimum :: (Foldable t, Ord a) => t a -> a
repeat :: a -> [a]
replicate :: Int -> a -> [a]
take, drop :: Int -> [a] -> [a]
splitAt :: Int -> [a] -> ([a], [a])
takeWhile, dropWhile :: (a -> Bool) -> [a] -> [a]
group :: Eq a => [a] -> [[a]]
inits, tails :: [a] -> [[a]]
elem, notElem :: (Foldable t, Eq a) => a -> t a -> Bool
lookup :: Eq a => a -> [(a, b)] -> Maybe b
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
filter :: (a -> Bool) -> [a] -> [a]
zip :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
nub :: Eq a => [a] -> [a]
delete :: Eq a => a -> [a] -> [a]
sort :: Ord a => [a] -> [a]
sortOn :: Ord b => (a -> b) -> [a] -> [a]
```

Maybe

```
data Maybe a = Nothing | Just a

maybe :: b -> (a -> b) -> Maybe a -> b
isJust, isNothing :: Maybe a -> Bool
listToMaybe :: [a] -> Maybe a
maybeToList :: Maybe a -> [a]
catMaybes :: [Maybe a] -> [a]
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
```

Typeclass Definitions

```
class Semigroup a where
  (<>)   :: a -> a -> a

class Semigroup m => Monoid m where
  mempty :: m

class  Show a  where
  show   :: a   -> String

class  Eq a  where
  (==)   :: a -> a -> Bool
  (/=)   :: a -> a -> Bool

class Eq a => Ord a where
  compare :: a -> a -> Ordering
  (<), (<=), (>=), (>) :: a -> a -> Bool
  max, min             :: a -> a -> a

class Functor f where
  fmap   :: (a -> b) -> f a -> f b

class Functor f => Applicative f where
  pure   :: a -> f a
  (<*>)  :: f (a -> b) -> f a -> f b

class Applicative m => Monad m where
  return :: a -> m a
  (>>=)  :: m a -> (a -> m b) -> m b

class Foldable t where
  foldMap :: Monoid m => (a -> m) -> t a -> m

class Arbitrary a where
  arbitrary :: Gen a
  shrink    :: a -> [a]

class Num a where
  (+), (-), (*)       :: a -> a -> a
  negate              :: a -> a
  abs                 :: a -> a
  signum              :: a -> a
  fromInteger         :: Integer -> a
```

Typeclass Definitions

```
class Semigroup a where
  (<>)   :: a -> a -> a

class Semigroup m => Monoid m where
  mempty :: m

class  Show a  where
  show   :: a   -> String

class  Eq a  where
  (==)   :: a -> a -> Bool
  (/=)   :: a -> a -> Bool

class Eq a => Ord a where
  compare :: a -> a -> Ordering
  (<), (<=), (>=), (>) :: a -> a -> Bool
  max, min             :: a -> a -> a

class Functor f where
  fmap   :: (a -> b) -> f a -> f b

class Functor f => Applicative f where
  pure   :: a -> f a
  (<*>)  :: f (a -> b) -> f a -> f b

class Applicative m => Monad m where
  return :: a -> m a
  (>>=)  :: m a -> (a -> m b) -> m b

class Foldable t where
  foldMap :: Monoid m => (a -> m) -> t a -> m

class Arbitrary a where
  arbitrary :: Gen a
  shrink    :: a -> [a]

class Num a where
  (+), (-), (*)       :: a -> a -> a
  negate              :: a -> a
  abs                 :: a -> a
  signum              :: a -> a
  fromInteger         :: Integer -> a
```

Prelude Functions and Datatypes

Booleans

```
data Bool = False | True

(&&) :: Bool -> Bool -> Bool
(||) :: Bool -> Bool -> Bool
not  :: Bool -> Bool
```

Lists

```
data [] a = [] | a : [a]

(++) :: [a] -> [a] -> [a]
head, last :: [a] -> a
tail, init :: [a] -> [a]

null   :: Foldable t => t a -> Bool
length :: Foldable t => t a -> Int
map :: (a -> b) -> [a] -> [b]
reverse :: [a] -> [a]
intersperse :: a -> [a] -> [a]
transpose :: [[a]] -> [[a]]
foldl  :: Foldable t => (b -> a -> b) -> b -> t a -> b
foldr  :: Foldable t => (a -> b -> b) -> b -> t a -> b
concat :: Foldable t => t [a] -> [a]
and, or  :: Foldable t => t Bool -> Bool
any, all :: Foldable t => (a -> Bool) -> t a -> Bool
sum, product :: (Foldable t, Num a) => t a -> a
maximum, minimum :: (Foldable t, Ord a) => t a -> a
repeat :: a -> [a]
replicate :: Int -> a -> [a]
take, drop :: Int -> [a] -> [a]
splitAt :: Int -> [a] -> ([a], [a])
takeWhile, dropWhile :: (a -> Bool) -> [a] -> [a]
group :: Eq a => [a] -> [[a]]
inits, tails :: [a] -> [[a]]
elem, notElem :: (Foldable t, Eq a) => a -> t a -> Bool
lookup :: Eq a => a -> [(a, b)] -> Maybe b
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
filter :: (a -> Bool) -> [a] -> [a]
zip :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
nub :: Eq a => [a] -> [a]
delete :: Eq a => a -> [a] -> [a]
sort :: Ord a => [a] -> [a]
sortOn :: Ord b => (a -> b) -> [a] -> [a]
```

Maybe

```
data Maybe a = Nothing | Just a

maybe :: b -> (a -> b) -> Maybe a -> b
isJust, isNothing :: Maybe a -> Bool
listToMaybe :: [a] -> Maybe a
maybeToList :: Maybe a -> [a]
catMaybes :: [Maybe a] -> [a]
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
```

Functors, Applicatives, and Monads

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$) :: Functor f => a -> f b -> f a
($>) :: Functor f => f a -> b -> f b

liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
(*>) :: Applicative f => f a -> f b -> f b
(<*) :: Applicative f => f a -> f b -> f a
when :: Applicative f => Bool -> f () -> f ()

(>>) :: Monad m => m a -> m b -> m b
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
filterM :: Monad m => (a -> m Bool) -> [a] -> m [a]
sequence  :: Monad m => [m a] -> m [a]
join :: Monad m => m (m a) -> m a
zipWithM :: Monad m => (a -> b -> m c) -> [a] -> [b] -> m [c]
foldM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m b
replicateM :: Applicative m => Int -> m a -> m [a]

liftM :: Monad m => (a1 -> r) -> m a1 -> m r
liftM2 :: Monad m => (a1 -> a2 -> r) -> m a1 -> m a2 -> m r
...
```

1 Hoare Rules

```
  { P } s1 { Q }
  { Q } s2 { R }
------------------  (hoare_seq)
{ P } s1; s2 { R }


----------------------- (hoare_asgn)
{Q [x := a]} x := a {Q}


  {P'} c {Q'}
P ==> P'    Q' ==> Q
-------------------- (hoare_consequence)
   {P} c {Q}

         {P &&  b} s1 {Q}
         {P && !b} s2 {Q}
-------------------------------- (hoare_if)
{P} if b { s1 } else { s2 } {Q}

         {P && b} s {P}
---------------------------- (hoare_while)
{P} while b { s } {P && !b}
```

Nikita Krupin

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i:int:=a.length;
  out:= 0;
  while (i>= 0) {
    out:= out-a[i];
    i:= i - 1;
  }
  return out;
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i : int := 0;
  out:= 0;
  while (i< a.length) {
    out:= out-a[i];
    i:= i - 1;
  }
  return out;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → b → Bool → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [b] → [b]

(e) `foldr (const l)`
Foldable t => (a → b → b) → b → t a → b

(f) `("42")`
a → String → (a, String)

(g) `(,) "42"`
a → String →
a → b b → c

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
a → Bool

(k) `fmap (fmap (+1)) [Nothing]`
(a → b) → f a → f b
[Nothing] → [Int]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\X -> repeat X`

(c) `a -> Maybe b`
`\a -> Just b`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\x y z -> repeat . x repeat y repeat z`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\x y z -> Just x Just y Just z`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\a -> foldr (zip a b) []`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`
`\X -> putStrLn X`

(j) `(a,b) -> (a -> b -> c) -> c`
`\x y -> x . y`

(k) `((a,b) -> c) -> c`
`\(x,y) -> foldr () c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = m*m*m && m > 0 }}
  y := 0;
{{ y = m*m*m }}
  var x := m * m;
{{ y = m*x }}
  var z := m;
{{ y = z*x }}
  while (z > 0) {
{{ y = z*x && z > 0 }}
{{ y + x = (z-1)*x }} ->>
    z := z - 1;
{{ y + x = z*x }}
    y := y + x;
{{ y = z*x }}
}
{{ y = z*x && !(z > 0) }} ->>
{{ y = m * m  * m}}
```

y < m
0 < m
z < m
z = m && y = z*m*m && !(z > 0)

0	z = m - 1 y = 0 + x, m*m
1	z = m - 1 - 1 y = 0 + x + x
2	z = m - 1 - 1 - 1 y = 0 + x + x + x

z = 0
y = x*z

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave _ [] = []
weave [] _ = []
weave xs ys = fold (\(x:xs) (y:ys) => x:acc) []
weave (x:xs) (y:ys) = concat [x,y] (weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = []
powerset xs =
```

idk Haskell
ME

M Sharma

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    var i := a.Length - 1
    out := 0;

    while i > 0 {
        out := out - a(i)
        i := i - 1;
    }

}
```

```
method f2(a:array<int>) returns (out:int) {
    var i := 0;
    out := 0;

    while i < a.Length {
        out := out - a(i);
        i := i + 1;
    }

}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`

a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`

a -> a -> String

(d) `\x l -> x : l ++ l ++ [x]`

a -> [a] -> [a]

(e) `foldr (const 1)`

[Int] -> Int

(f) `(,"42")`

a -> (a, String)

(g) `(,) "42"`

(h) `reverse . reverse`

[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`

[Int] -> [Int]

(j) `let f x = x in (f 'a', f True)`

ill-typed

(k) `fmap (fmap (+1)) [Nothing]`

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`
(+2)

(b) `Bool -> [Bool]`

```
\x -> if x then [x] else [False]
```

(c) `a -> Maybe b`

```
\x -> Just x
```

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

```
zipWith (\x y -> show x == show y) [1,2,3] ['2','6','c']
```

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

```
\f a b -> zipWith f (maybeToList a) (maybeToList b)
```

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

```
\x (y:ys) -> if x == y then [x] else ys
```

(i) `Show a => [a] -> IO String`

```
\x -> do putStr $ reverse x
```

(j) `(a,b) -> (a -> b -> c) -> c`

```
\(x,y) f -> f x y
```

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = m * m * m                                  }}
  y := 0;
{{ y = m * m * m                                  }}
  var x := m * m;
{{ y = m * x                                      }}
  var z := m;
{{ y = z * x                                      }}
  while (Z > 0) {
{{ y + . . = z * x  && z > 0                      }} ->>
{{ y + x = (z - 1) * x                            }}
    z := z - 1;
{{ y + x = z * x                                  }}
    y := y + x;
{{ y     = z * x                                  }}
}
{{ y = z * x                          && !(z > 0) }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] [] = []
weave (x:xs) (y:ys) = [x] ++ [y] ++ weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

Samuel Kim 117827446

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i:int := a.Length - 1;
  out := 0;
  while (i >= 0) {
    out := out - a[i];
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i:int := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer: `a -> a`
`a -> a`

(b) `\x y -> (x,y)` a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
x → y → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)` ? ? acc lst
a → [Int] → a

(f) `(."42")`
(Maybe a, String)

(g) `(.) "42"`
b → (String, b)

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[a] → [[a]]

(j) `let f x = x in (f 'a', f True)`
(a → b) → a

(k) `fmap (fmap (+1)) [Nothing]`
fmap Maybe a

[Maybe Int]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

```
\x → x + 3
```

(b) `Bool -> [Bool]`

```
\b → False : [b]
```

(c) `a -> Maybe b`

```
\a → Nothing
```

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

```
\ f (h:t) (c:cs) → if c == 'k' then [false && f (h+3) c] else [false]
```

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

```
\f a b → Just (f a b)
```

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

```
\f f2 (a:as) → let x = f a in let b = ((f2 x) && false) in [x]
```

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

```
\a f → let x = head (f a) in [(a, x)]
```

(h) `Eq a => a -> [a] -> [a]`

```
\a alist → if a == (head alist) then [a] else [a]
```

(i) `Show a => [a] -> IO String`

```
\(h:t) → putStrLen "hi"
```

(j) `(a,b) -> (a -> b -> c) -> c`

```
\(a,b) f → f a b
```

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z >=0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

m = 3 y = 0 x = 9 z = 3
m = 3 y = 9 x = 9 z = 2
m = 3 y = 18 x = 9 z = 1
m = 3 y = 27 x = 9 z = 0

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m ≥ 0 && 0 = pow(m * m, m-z)                          }}
  y := 0;
{{ m ≥ 0 && y = pow(m * m, m-z)                          }}
  var x := m * m;
{{ m ≥ 0 && y = pow(x, m-z)                              }}
  var z := m;
{{ z ≥ 0 && y = pow(x, m-z)                              }}
  while (Z > 0) {
{{ z ≥ 0 && y = pow(x, m-z) && (z > 0)                   }} ->>
{{ z-1 ≥ 0 && y+x = pow(x, m-z)                          }}
    z := z - 1;
{{ z ≥ 0 && y+x = pow(x, m-z)                            }}
    y := y + x;
{{ z ≥ 0 && y = pow(x, m-z)                              }}
}
{{ z ≥ 0 && y = pow(x, m-z)              && !(z>0)       }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave lst lst2 = case (lst, lst2) of
  ([],[]) -> []
  ([], lst2) -> []
  (lst, []) -> []
  (h:t, h2:t2) -> h:[h2] (++) (weave t t2)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset lst = case lst of
  [] -> []
  h:t -> lst (++) [h] (++) powerset t
```

Sai Chanda 118612393

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0;
  var i : int := a.Length-1;
  while i >=0
    invariant -1<=i<a.Length
  { out := out - a[i];
    i := i-1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  out := 0;
  var i : int := 0;
  while i < a.Length
    invariant 0<=i<=a.Length
  {
    out := out - a[i];
    i := i+1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a→b→(a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a→a→String

(d) `\x l -> x : l ++ l ++ [x]`
a→[a]→[a]

(e) `foldr (const l)`
(a→b→b)→b→ta→b

(f) `(:"42")`
ill-typed

(g) `(,) "42"`
a→(String, a)

(h) `reverse . reverse`
ill typed

(i) `\l -> filter (< 42) (l ++ l)`
[Int]→[Int]

(j) `let f x = x in (f 'a', f True)`
(Char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`
ill typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [True] else [False]`

(c) `a -> Maybe b`
`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\fun (x:xs) (y:ys) -> if (x) == 0 && (y) == 'c' then [fun x y] else [False]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\fun a b -> case of a Nothing -> Nothing Just a -> case of b Nothing -> Nothing Just b -> let bc = fun a b in case of bc Nothing -> Nothing Just bc -> bc`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`func fun boo (x:xs) = if boo (fun x) then [fun x] else []`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`
`funcEq a (x:xs) = if eq a x then a : funceq a xs else funceq a x>`

(i) `Show a => [a] -> IO String`
`\s (x:xs) -> s x`

(j) `(a,b) -> (a -> b -> c) -> c`
`func(a, b) f = f a b`

(k) `((a,b) -> c) -> c`
`func f = (f())`

`func a conv = case a of Nothing -> []`
`Just a -> let b = conv a in case b of [] -> []`
`(x:xs) -> [(a, x)]`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

→ invariant $0 \leq z \leq m$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 == (m-m)*m*m && m * m == m*m }}
  y := 0;
{{ y == (m-m)*m*m && m*m == m*m }}
  var x := m * m;
{{ y == (m-m)*m*m && m*m == m*m }}
  var z := m;
{{ y == (m-z)*m*m && x == m*m }}
  while (z > 0) {
{{ y == (m-z)*m*m && z >= 0 && x == m*m }} ->>
{{ y+x == (m-z-1)*m*m && z-1 >= 0 && x == m*m }}
    z := z - 1;
{{ y+x == (m-z)*m*m && z >= 0 && x == m*m }}
    y := y + x;
{{ y == (m-z)*m*m && z >= 0 && x == m*m }}
}
{{ y == (m-z)*m*m && !(z > 0) && x == m*m }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the ~~same length~~.

```
weave :: [int] -> [int] -> [int]
weave [] [] = []
weave (x:xs) (y:ys) = [x,y] (++) weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [int] -> [[int]]
powerset [] = [[]]
powerset (x:xs) = (x:xs) : (powerset (para delete (x:xs) (x:xs)))
```

// Iterate through the list deleting one element, then call powerset on that, and delete another element and powerset gets called again.

Sachin Mansukhani

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i: int := a.len - 1;
  out := 0;
  while (i >= 0) {
    out := out - a[i];
    i := i + 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i: int := 0;
  out := 0;
  while (i < a.len) {
    out := out - a[i];
    i := i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
Eq a ⇒ a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a]

(e) `foldr (const 1)`
ill-typed

(f) `(,"42")`
(a → b → (a,b), String)

(g) `(,) "42"`
a → (String, a)

(h) `reverse . reverse`
[a] → [a] → ill-typed

(i) `\l -> filter (< 42) (1 ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`
(a → b) → f a → f b
ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [x] else []`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`foo f is cs -> (f (head is) (head cs)) : []`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`f a b -> Nothing`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`foo f1 f2 xs -> let b = f1 (head xs) in if f2 b then [b] else []`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
undefined

(h) `Eq a => a -> [a] -> [a]`
`foo _ [] = []`
`foo y (x:xs) = if x == y then [x] else foo xs`

(i) `Show a => [a] -> IO String`
Undefined

(j) `(a,b) -> (a -> b -> c) -> c`
`foo (a,b) f = f a b`

(k) `((a,b) -> c) -> c`
Undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Cube of 2
x = 4 y = 0 m = 2
z = 2

first: z = 1, y = 4
second: z = 0, y = 8

Cube of 3
x = 9 y = 0 m = 3
z = 3

first: z = 2, y = 9
second: z = 1, y = 18
third: z = 0, y = 27

y = x · (m − z)

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ m ≥ 0 && 0 == 0 && m·m == m·m }}
  y := 0;
{{ m ≥ 0 && y == 0 && m·m == m·m }}
  var x := m * m;
{{ m ≥ 0 && y == 0 && x == m·m }}
  var z := m;
{{ z ≥ 0 && y == x·(m−z) && x == m·m }}
  while (z > 0) {
{{ z ≥ 0 && y == x·(m−z) && x == m·m && z > 0 }} ->>
{{ z−1 ≥ 0 && y+x == x·(m−z+1) && x == m·m }}
    z := z - 1;
{{ z ≥ 0 && y+x == x·(m−z) && x == m·m }}
    y := y + x;
{{ z ≥ 0 && y == x·(m−z) && x == m·m }}
  }
{{ z ≥ 0 && y == x·(m−z) && x == m·m && z ≤ 0 }} ->>
{{ y = m * m  * m}}
```

Invariants =
z ≥ 0
y = x · (m − z)
x = m · m

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave : [a] -> [a] -> [a]
weave [] _ = []
weave _ [] = []
weave (x:xs) (y:ys) = (x:y: weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]
powerset (x:xs) = ([x]: (x:xs): powerset xs)
```

Aditya
Menachery

UID: 118765723

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  requires a.Length ≥ 0
  out:int := 0;
  var i:int := a.Length-1;
  while (i ≥ 0)
    invariant 0≤i<a.Length
  {
    out := out - a[i];
    i := i-1;
  }
  return out;
}
```

```
method f2(a:array<int>) returns (out:int) {
  requires a.Length≥0
  out:int := 0;
  var i:int := 0;
  while (i<a.Length)
    invariant 0≤i<a.Length
  {
    out := out - a[i];
    i := i+1;
  }
  return out;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
a → [b] → a

(f) `(.)"42"`
(Nothing, String)

(g) `(.) "42"`
ill-typed

(h) `reverse . reverse`
a → a

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`
ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix. do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\a -> if a then [a] else [a]`

(c) `a -> Maybe b`
`fun a = Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\x y z -> zipWith x y z`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`fun f (Just a) (Just b) = Just f a b`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`fun x y (z:zs) = if y (x z) then x z else x z`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`fun (Just a) f = [(a, head (f a))]`

(h) `Eq a => a -> [a] -> [a]`
`fun a b = [(==) a (head b)]`

(i) `Show a => [a] -> IO String`
`fun a = show (head a)`

(j) `(a,b) -> (a -> b -> c) -> c`
`fun (x,y) z = z x y`

(k) `((a,b) -> c) -> c`
`undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{    0=(m-m)·m·m  ∧∧ m≥0                    }}
  y := 0;
{{    y=(m-m)·m·m  ∧∧ m≥0                    }}
  var x := m * m;
{{    y=(m-m)·x    ∧∧ m≥0                    }}
  var z := m;
{{    y=(m-z)·x    ∧∧ z≥0                    }}
  while (z > 0) {
{{    y=(m-z)·x ∧∧ z>0 ∧∧ z≥0               }} ->>
{{  y+x=(m-(z-1))·x ∧∧ z≥0                   }}
    z := z - 1;
{{    y+x=(m-z)·x  ∧∧ z≥0                    }}
    y := y + x;
{{    y=(m-z)·x    ∧∧ z≥0                    }}
  }
{{ y=(m-z)·x ∧∧ !(z>0) ∧∧ z≥0                }} ->>
{{ y = m * m  * m}}
```

z	y	x
m	0	m·m
m-1	m·m	m·m
m-2	2m·m	m·m

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave a [] = a
weave [] b = b
weave (x:xs) (y:ys) = x:[y] ++ (weave xs ys)
weave [] [] = []
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset [] = [[]]
powerset (x:xs) = (helper x (powerset xs))
```

```
helper :: a -> [[a]] -> [[a]]
helper a b = b ++ (map (a:) b)
```

Shril Shah
118915241

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
    var i:int := a.Length-1;
    out := 0;
    while (i >= 0)
        invariant i <= a.Length-1
        invariant i >= -1;
    {
        out := out - a[i];
        i := i-1;
    }
}
```

```
method f2(a:array<int>) returns (out:int) {
    var i:int := 0;
    out := 0;
    while (i < a.Length)
        invariant i >= 0
        invariant i <= a.Length
    {
        out := out - a[i];
        i := i+1;
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
[a] → a

(f) `(,"42")`
ill-typed

(g) `(,) "42"`
a → (String, a)

(h) `reverse . reverse`
(a → [b]) → a → [b]

(i) `\l -> filter (< 42) (l ++ l)`
[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`
f(a→b) → b → bool

(k) `fmap (fmap (+1)) [Nothing]`
[Int] → Nothing

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [true] else [false]`

(c) `a -> Maybe b`
`func x = if x == x then Nothing else Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`func f (x:xs) (y:ys) = if x > 0 && y == 'a' then [f x y] else [False]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`func f Just x Just y -> f x y`
`func _ _ _ -> Nothing`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`func f g (x:xs) = if (g (f x)) then [(f x)] else [(f x)]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`func Just x f = case of (f a) where (y:ys) -> [(x, y)]`

(h) `Eq a => a -> [a] -> [a]`
`func x (y:ys) = if (Eq x y) then [x] else [y]`

(i) `Show a => [a] -> IO String`
`func (x:xs) = show x`

(j) `(a,b) -> (a -> b -> c) -> c`
`func (x, y) f = f x y`

(k) `((a,b) -> c) -> c`
`func f = f Nothing Nothing g`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = (m-m)*m*m  && m>0                                  }}
  y := 0;
{{ y = (m-m)*m*m  && m>0                                  }}
  var x := m * m;
{{ y = (m-m)*m*m  && m>0                                  }}
  var z := m;
{{ y = (m-z)*m*m  && z>0                                  }}
  while (z > 0) {
{{ y = (m-z)*m*m  && z>0                                  }} ->>
{{ y+x = (m-(z-1))*m*m  && z-1>=0                         }}
    z := z - 1;
{{ y+x = (m-z)*m*m  && z>=0                               }}
    y := y + x;
{{ y = (m-z)*m*m  && z>=0                                 }}
}
{{ y = (m-z)*m*m  && !(z>0) && z>=0                       }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] → [a] → [a]
weave [] [] = []
weave (x:xs) (y:ys) = [x] ++ [y] ++ (weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] → [[a]]
powerset [] = [[]]
powerset [x] = [[x]]
powerset (x:xs) = powerset x ++ powerset xs ++ [(x:xs)] ++ powerset (helper x:xs)

helper [a] → [[a]]
helper (x:(y:ys)) = [[]] ++ [x:(y:ys)] ++ helper x ys
```

Jason Zhong

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

[1,2,3]

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0;
  var index:int := a.Length-1;
  while (index >= 0) {
    out := a[index]-out;
    index := index-1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  out := 0;
  var index: int := 0;
  while (index < a.Length) {
    out := out - a[index];
    index := index+1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`

a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`

a -> a -> String ?

(d) `\x l -> x : l ++ l ++ [x]`

a -> [a] -> [a]

(e) `foldr (const 1)`

ill-typed

(f) `(,"42")`

ill-typed

(g) `(,) "42"`

a -> (String, a)

(h) `reverse . reverse`

[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`

[a] -> [a]

(j) `let f x = x in (f 'a', f True)`

(Char, Bool)

(k) `fmap (fmap (+1)) [Nothing]`

ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [x == True]`

(c) `a -> Maybe b`
`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f x y -> (zipWith f (3:x) ('a':y)) : [True]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f x y -> listToMaybe (zipWith (maybeToList x) (maybeToList y))`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f1 f2 x -> filter f2 (map f1 x)`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\x f -> zip (maybeToList x) (map f (maybeToList x))`

(h) `Eq a => a -> [a] -> [a]`
undefined

(i) `Show a => [a] -> IO String`
undefined

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`
undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules -- write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 == m*m*m - m*m*m && m >= 0 && m*m == m*m }}
  y := 0;
{{ y == m*m*m - m*m*m && m >= 0 && m*m == m*m }}
  var x := m * m;
{{ y == m*m*m - x*m && m >= 0 && m*m == m*m }}
  var z := m;
{{ y == m*m*m - x*z && z >= 0 && x == m*m }}
  while (z > 0) {
{{ y == m*m*m - x*z && z >= 0 && z > 0 && x == m*m }} ->>
{{ y + x == m*m*m - (x * (z-1)) && z - 1 >= 0 && x == m*m }}
    z := z - 1;
{{ y + x == m*m*m - x*z && z >= 0 && x == m*m }}
    y := y + x;
{{ y == m*m*m - x*z && z >= 0 && x == m*m }}
}
{{ y == m*m*m - x*z && z >= 0 && !(z > 0) && x == m*m }} ->>
{{ y == m * m  * m}}
```

0 m*m m
m*m m*m m-1
2*m*m m*m m-2 m*m*m

y = m*m*m - x*z

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] [] = []
weave [] l2 = l2
weave l1 [] = l1
weave (x:xs) (y:ys) = x:y:(weave xs ys)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset [] = [[]]
powerset (x:xs) = (map (y -> x:y)(powerset xs)) ++ powerset xs
```

AKSHIT SANORIA

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0
  i := 0
  while i < a.length {
    out := a[i] - out
    i := i + 1
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  out := 0
  i := 0
  while i < a.length {
    out := out - a[i]
    i := i + 1
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> a -> a | (a, a)

(d) `\x l -> x : l ++ l ++ [x]`
a -> [a] -> [a]

(e) `foldr (const 1)`
(a -> b -> a) -> a] -> a | [a]

(f) `(,"42")`
ill type

(g) `(,) "42"`
ill type

(h) `reverse . reverse`
ill type

(i) `\l -> filter (< 42) (1 ++ l)`
a -> [a] -> [a]

(j) `let f x = x in (f 'a', f True)`
(a -> b) -> c -> (d, e)

(k) `fmap (fmap (+1)) [Nothing]`
a -> (a -> b) -> [ ] -> b ill type.

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

```
if x then [true, true] else [false, false]
```

(c) `a -> Maybe b`

```
check a = Nothing
check a = Just a
```

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

```
fun f (a:at) (b:bt) = f a b : fun f at bt
```

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

```
f a b c) ma mb =
```

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

```
fun f g (x:xt) = g (f x) : fun f g xt
```

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

```
fun a f b =
```

(h) `Eq a => a -> [a] -> [a]`

```
let a (x:xt) = x in a x : a xt
```

(i) `Show a => [a] -> IO String`

```
a = do "Hello"
```

(j) `(a,b) -> (a -> b -> c) -> c`

```
f x g = g x
```

(k) `((a,b) -> c) -> c`

```
let f x = x in f x
```

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules---write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 == 0 * 0 * m                                  }}
  y := 0;
{{ y == 0 * m * m                                  }}
  var x := m * m;
{{ y == m * x                                      }}
  var z := m;
{{ y == z * x                                      }}
  while (z > 0) {
{{ y == z * x && z > 0                             }} ->>
{{ (y + x) == (z - 1) * x                          }}
    z := z - 1;
{{ (y + x) == z * x                                }}
    y := y + x;
{{ y == z * x                                      }}
}
{{ y == z * x                  && !(z > 0)         }} ->>
{{ y = m * m * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length

```
weave _ [] = []
weave [] _ = []
weave (x:xt) (y:yt) = [x,y] ++ weave xt yt
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = []
powerset (x:xt) = foldr (:) x : powerset xt
```

Vijay Ved 1187 48344

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length-1;
  out := 0;
  while (i >= 0) {
    out := out - a[i];
    i := i-1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i+1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

(e) `foldr (const 1)`
[a] → b → b

(f) `(,"42")`
(a, String)

(g) `(,) "42"`
ill-typed

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`
[a] → [a]

(j) `let f x = x in (f 'a', f True)`
ill-typed

(k) `fmap (fmap (+1)) [Nothing]`
ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

```
\x -> if x then [true] else [false]
```

(c) `a -> Maybe b`

```
\a -> Nothing
```

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

```
zipWidth
```

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

```
\f1 f2 l -> if f2 (f1 head l) then [f1 heal l] else [f1 head l]
```

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

```
\x f -> [Just x, head(f x)]
```

(h) `Eq a => a -> [a] -> [a]`

```
delete a
```

(i) `Show a => [a] -> IO String`

```
f l = show(head l)
```

(j) `(a,b) -> (a -> b -> c) -> c`

```
\(a,b) f -> f a b
```

(k) `((a,b) -> c) -> c`

```
fun f -> f (a,b)
```

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ y = 0 && (z >= 0)                                  }}
  y := 0;
{{ y = m*m-(m-m) && (z >= 0)                          }}
  var x := m * m;
{{ y = x*(m-m) && (z >= 0)                            }}
  var z := m;
{{ y = x*(m-z) && (z >= 0)                            }}
  while (Z > 0) {
{{ y = x*(m-z) && z > 0 && (z > 0)                     }} ->>
{{ y + x = x*(m-(z-1)) && (z-1 >= 0)                  }}
    z := z - 1;
{{ y + x = x*(m-z) && (z >= 0)                        }}
    y := y + x;
{{ y = x*(m-z) && (z >= 0)                            }}
}
{{ (y = x*(m-z) && (z >= 0)) && ! (z > 0)             }} ->>
{{ y = m * m * m}}
```

$y = x + m(m-z)$

81

$y = x(m-z)$

9

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

```
weave :: [a] -> [a] -> [a]
```

You can assume that the lists have the same length.

```
weave (x:xs) (y:ys) = x : y : weave xs ys
weave _ _ = []
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [Int] -> [[Int]]
powerset [] = []
powerset [a] = [a]
powerset (x:xs) = [xs] :powerset x : powerset xs
```

Recursively adds each list until down to just one element

Felix Gu
117831085

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length - 1
  out := 0;
  while (i >= 0)
  {
    out := out - a[i];
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while (i < a.Length)
  {
    out := out - a[i];
    i := i + 1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> b -> (Bool -> show x -> show (a, b)

(d) `\x l -> x : l ++ l ++ [x]`
a -> a -> a -> [a] -> [a] -> [a] -> [a] -> [a] -> [a] -> [a] -> [a]

(e) `foldr (const 1)`
Int -> Int -> Int

(f) `(,"42")`
a -> String -> (a, String)

(g) `(,) "42"`
(a -> b -> (a, b)) -> String

(h) `reverse . reverse`
[a] -> [a]

(i) `\l -> filter (< 42) (l ++ l)`
a -> (a -> a -> Bool) -> [a] -> [a]

(j) `let f x = x in (f 'a', f True)`
(a -> b) -> (b, b)

(k) `fmap (fmap (+1)) [Nothing]`

((Int -> Int) -> Int -> Int) -> f [Maybe a] -> f [Maybe b]

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [True]`

(c) `a -> Maybe b`
`\x -> Just b`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`f [5, 4] ['c', 'd'] = True`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`f a b c = Just a Just b Just c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`fun (f1 x) (f2 y) [x] = [y]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`Just x (f x) = [(x, f x)]`

(h) `Eq a => a -> [a] -> [a]`
`(==) [==] = [==]`

(i) `Show a => [a] -> IO String`
`show [a] = putStrLn a`

(j) `(a,b) -> (a -> b -> c) -> c`
`\x (a,b) -> f a b`

(k) `((a,b) -> c) -> c`
`f (a, b)`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{                                                    }}
  y := 0;
{{                                                    }}
  var x := m * m;
{{                                                    }}
  var z := m;
{{                                                    }}
  while (z > 0) {
{{                                                    }} ->>
{{                                                    }}
    z := z - 1;
{{                                                    }}
    y := y + x;
{{                                                    }}
}
{{                                                    }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave _ [] = []
weave [] _ = []
weave (x:xs) (y:ys) = x : y : weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset :: [a] -> [[a]]
powerset [] = [[]]
powerset [a] = [a] : [[]]
powerset [a,b] = [a,b] : [a] : [b] : [[]]
powerset (z = x:y:xs) =
  z : [] : [x] : [y] : [x,y] : [x,xs] : [y,xs] : powerset xs
```

Tyler Kravath
117278637

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0;
  var y:Int := a.Length-1;
  while (y >= 0) {
    out := a[y] - out;
    y := y-1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  out := 0;
  var y:Int := 0;
  while (y < a.Length) {
    out := out - a[y]
    y := y+1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`

(c) `\x y -> if x == y then show x else show (x,y)`

(d) `\x l -> x : l ++ [x]`

(e) `foldr (const 1)`

(f) `(.".42")`

(g) `(,".42")`

(h) `fmap (fmap (+1)) [Nothing]`

(i) `\l -> filter (< 42) (1 ++ l)`

(j) `let f x = x in (f 'a', f True)`

(k) `fmap (fmap (+1))`

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [x]`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f (x:xs) (1:1ss) -> [f x 1]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f Just a Just b -> Just f a b`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f1 f2 (x:xs) -> map f1 xs`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`Just a f -> [(a, f a)]`

(h) `Eq a => a -> [a] -> [a]`
`nub`

(i) `Show a => [a] -> IO String`
`putStrLn a`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`
`\f (a,b)`

`(.) :: (b -> c) -> (a -> b) -> a -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
    {
      z := z - 1;
      y := y + x;
    }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }}
{{ 0 + m*m*m = m*m*m }}
y := 0;
{{ y + m*m*m = m*m*m }}
var x := m * m;
{{ y + x*m = m*m*m }}
var z := m;
{{ y + x*z = m*m*m }}
while (z > 0)
{{ y + x*z = m*m*m && z > 0 }}
{
  {{ y + x*(z-1) + x = m*m*m }}
  z := z - 1;
  {{ y + x*z + x = m*m*m }}
  y := y + x;
  {{ y + x*z = m*m*m }}
}
{{ y + x*z = m*m*m && !(z > 0) }}
{{ y = m*m*m }}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [a] _ = [a]
weave _ [b] = [b]
weave (x:xs) (ls:lss) = x : (ls : weave xs lss)
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
Powerset _ = [[]]
Powerset (x:xs) = Powerset xs ++ [x : subset | subset ++ Powerset xs]
```

Michael Howe
117094091

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i: int = a.Length - 1;
  out := 0;
  while (i >= 0){
    out = out - a[i];
    i := i-1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i:int = 0;
  out := 0;
  while(i<a.Length){
    out = out - a[i];
    i := i+1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a -> b -> (a, b)

(c) `\x y -> if x == y then show x else show (x,y)`
a -> b -> String

(d) `\x l -> x : l ++ l ++ [x]`
a -> a -> [a]

(e) `foldr (const 1)`
a -> b -> a -> b -> [a] -> b

(f) `(."42")`
undefined

(g) `(,) "42"`
String -> b -> (String, b)

(h) `reverse . reverse`
a -> a

(i) `\l -> filter (< 42) (l ++ l)`
int -> [int]

(j) `let f x = x in (f 'a', f True)`

(k) `fmap (fmap (+1)) [Nothing]`

a ->

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\True -> [True]`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`zipWith (:) (1) ("a")`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\a -> Just c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\a -> takeWhile a [a]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\Just a ->`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`
`\a -> putStrLn "tag"`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) ->`

(k) `((a,b) -> c) -> c`
`\(a,b) ->`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with `->>`. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{                                  }}
  y := 0;
{{                                  }}
  var x := m * m;
{{                                  }}
  var z := m;
{{                                  }}
  while (z > 0)
  {{                                  }} ->>
  {{                                  }}
  {{                                  }}
    z := z - 1;
  {{                                  }}
    y := y + x;
  {{                                  }}
  }
{{ y = m * m * m }}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave (x:xs) (y:ys) = x : y : xs
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists. including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]
powerset (x:xs) = powerset xs ++ [x : subset | subset ← powerset xs]
```

Advik Sachdeva
118582597

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length-1
  out := 0;
  while(i>=0 && i<a.Length) {
    out:=out-a[i];
    i:= i-1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out:=0;
  while(i<a.Length){
    out:= out-a[i];
    i:=i+1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`
a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`
a → a → String

(d) `\x l -> x : l ++ l ++ [x]`
a → [a] → [a]

const :: a → b → b (const b) (is type a

(e) `foldr (const 1)`
[a] → b → b

(f) `(,"42")`
(a, String)

(g) `(,) "42"`
ill typed

(h) `reverse . reverse`
[a] → [a]

(i) `\l -> (filter (< 42)) (1 ++ l)`
[a] → [a]

[a]

<42

(a → Bool) → ([a] → [a])

(j) `let f x = x in (f 'a', f True)`
ill typed

(k) `fmap (fmap (+1)) [Nothing]`
ill typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [x] else [x]`

(c) `a -> Maybe b`
`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f xy -> 1:x ++ 'h':y ++ if f (head x) (head y) then [f] else [f]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`zipWith`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f f2 xs -> if f2 (f (head xs)) then [f (head xs)] else [f (head xs)]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\x f -> [Just a, (head (f (Just a)))]`

(h) `Eq a => a -> [a] -> [a]`
`delete a`

(i) `Show a => [a] -> IO String`
`f x = show (head x)`

(j) `(a,b) -> (a -> b -> c) -> c`
`met x f = case x of (a,b) -> f a b`

(k) `((a,b) -> c) -> c`
`\f -> f (a, b)`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

iteration	x	z	y	m
0	9	2	9	3
1	9	1	18	3
2	9	0	27	3

$(m-z)\cdot x = y$ $\quad \& \& \; x = m*m$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0 = m*m(m-m) && m*m = m*m && m>0 }}
  y := 0;
{{ y = m*m(m-m) && m*m = m*m && m>0 }}
  var x := m * m;
{{ y = x(m-m) && x = m*m && m>0 }}
  var z := m;
{{ y = x(m-z) && x = m*m && m>0 }}
  while (z > 0) {
{{ y = x(m-z) && x = m*m && m>0 && z>0 }} ->>
{{ y+x = x(m-z+1) && x = m*m && m>0 }}
    z := z - 1;
{{ y+x = x(m-z) && x = m*m && m>0 }}
    y := y + x;
{{ y = x(m-z) && x = m*m && m>0 }}
}
{{ y = x(m-z) && x = m*m && m>0 && !(z>0) }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] [] -> []
weave (x:xs) (y:ys) -> x:y:weave xs ys
weave _ _ -> []
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset [] = [[]]
powerset arr = arr : helper arr : powerset (tail arr) where
    helper ar = reverse (tail (reverse arr))
-- The helper will give more subarrays. for example
-- [1,2,3] would be [3,2,1] then the tail so [2,1] then
-- reverse so [1,2] would be added
```

Van Lin

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

parsers int to int

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
var i:int := 0;
out := 0;
while (i < a.length){
out := a[a.length - i] - out;
i := i + 1;
}
return out
}
```

```
method f2(a:array<int>) returns (out:int) {
var i : int := a.length - 1;
out := 0;
while (i < a.length){
out := a[i] + a[i-1] + out;
i := i - 2;
}
return out
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`
Example answer:
`a -> a`

(b) `\x y -> (x,y)`

a → b → (a,b)

(c) `\x y -> if x == y then show x else show (x,y)`

ill-typed

(d) `\x l -> x : l ++ l ++ [x]`

a → [a] → [a]

(e) `foldr (const 1)`

a → [a] → Int

(f) `(,"42")`

(t1, String)

(g) `(,) "42"`

ill-typed

(h) `reverse . reverse`

[a] → [a]

(i) `\l -> filter (< 42) (l ++ l)`

[Int] → [Int]

(j) `let f x = x in (f 'a', f True)`

ill-typed

(k) `fmap (fmap (+1)) [Nothing]`

f (f [Int])

[int] -> int

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`

\ x -> repeat show

(b) `Bool -> [Bool]`

repeat true

(c) `a -> Maybe b`

\ a -> Just C \f -> f a)

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
a b c a b c

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

\ f a b -> Just C f _ g) a)

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

\ a -> a : b

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

\ (a, b) -> c, b -> a

(k) `((a,b) -> c) -> c`

\ f -> int

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with ->>. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{{ m > 0 }} ->>
{{ 0=0 , &&  m==m                                  }}
  y := 0;
{{y=0 && m==m                                      }}
  var x := m * m;
{{ y=0 && x==m*m && m==m                           }}
  var z := m;
{{x==m*m && x+(y+x) == m*m*m && z>=0                }}
  while (Z > 0) {
{{ x==m*m && x+(y+z) == m*m+m*m                     }} ->>
{{ x==m*m && x+(y+x) == m*m*m                       }}
    z := z - 1;
{{  x==m*m && x+y == m*m*m                         }}
    y := y + x;
{{x==m*m && x+y == m*m*m && (z>=0)                  }}
}
{{ x==m*m  && x+y == m*m*m && (z-1)                 }} ->>
{{ y = m * m  * m}}
```

Question 5 (20 points)

Implement the following Haskell functions:

(a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave _ [] = []
weave [] _ = []
weave (x:xs) (y:xs) = x:y++ weave xs ys
```

(b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
```

```
powerset _ [] = []
powerset [] _ = []
powerset (x:xs) (y:xs) = x:y++ powerset ys xs
```